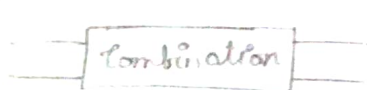
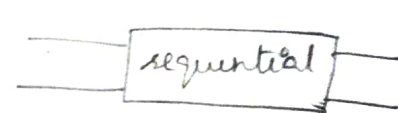


FSM - finite state machine

It is an abstract model for describing sequential circuits.

combinational circuit	sequential circuit
Output - present input	Output - present input and input
Eg: adder	Eg: counter
	

Two types of FSM,

1) Mealy machine

→ Output depends on both input and present state

2) Moore machine

→ Output depends on present state

Steps for Designing FSM

- * Define a problem
- * Draw a state transition diagram
- * Draw a state table
- * Remove redundant states
- * State assignment
- * Find the flip flop for realization of circuit and draw excitation table
- * Find boolean function for flipflop input
- * Draw the logic diagram.

Sequence Detector

- overlapping sequence detector
- non-overlapping sequence detector.

Eg: 1010

Input → 1010010001010

overlap → 00010000000001

S.D

Eg: 1011

↑ Initial state for overlap

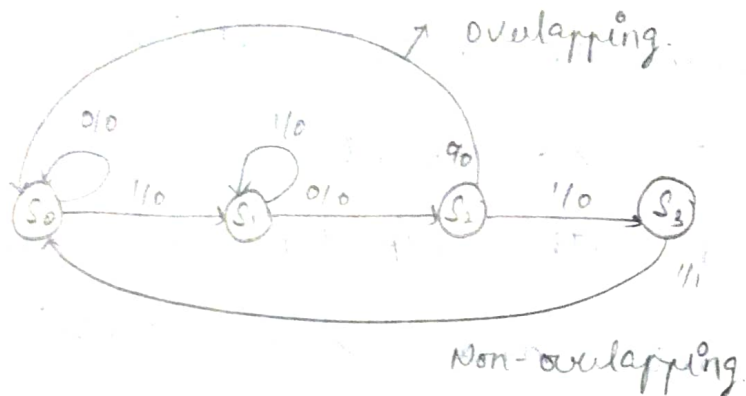
Input : 1011011001010

overlap S.D: 00010010000000

Non-overlap: 00010000000000

since that 1 is not considered as
sequence is not detected. initial

State transition diagram



*The states should be minimal.

State table

x	Present state	Next state	Output
0	S ₀ 0 0	0 0	0
1	S ₀ 0 0	0 1	0
0	S ₁ 0 1	1 0	0
1	S ₁ 0 1	0 1	0
0	S ₂ 1 0	0 0	0
1	S ₂ 1 0	1 1	0
0	S ₃ 1 1	1 0	0
0	S ₃ 1 1	0 0	1

Flipflop Realization.

D flipflop is used

Next state is the input

In D flipflop, input is same as output

Excitation Table

x	Present state		D ₀	D ₁	Output
0	0	0	0	0	0
1	0	0	0	1	0
0	0	1	1	0	0
1	0	1	0	1	0
0	1	0	0	0	0
1	1	0	1	1	0
0	1	1	1	0	0
1	1	1	0	0	1

Boolean expression.

K-map for D₁:

	Q ₁ Q ₀			
x	00	01	11	10
0		1	1	
1				1

$$D_1 = \bar{x}Q_0 + Q_1\bar{Q}_0x$$

K-map for D₀:

	Q ₁ Q ₀			
x	00	01	11	10
0				
1	1	1		1

$$D_0 = x\bar{Q}_1 + xQ_1Q_0$$

K-map for Y:

	Q ₁ Q ₀			
x	00	01	11	10
0				
1			1	

$$Y = xQ_1Q_0$$

```

module sd(clk, rst, y, x);
    input clk, rst, x;
    output reg y;
    reg [1:0] ps, ns;
    parameter s0 = 2'b00, s1 = 2'b01, s2 = 2'b10, s3 = 2'b11;
    always @(ps, x)
    begin
        case (ps)
            s0: begin
                    if (x == 1'b0) ns = s0; y = 1'b0; end
                    else
                        begin ns = s1; y = 1'b0; end
                    end.
            s1: begin
                    if (x == 1'b0) begin
                        ns = s2;
                        y = 1'b0;
                    end
                    else
                        begin
                            ns = s1;
                            y = 1'b0;
                        end
                    end.
            s2: begin
                    if (x == 1'b0)
                        begin
                            ns = s0;
                            y = 1'b0;
                        end
                    else
                        begin
                            ns = s3

```



```

        y = 1'b0;
    end
end
S3: begin
    if (x == 1'b0)
        begin
            ns = S2;
            y = 1'b0;
        end
    else
        begin
            ns = S0;
            y = 1'b1;
        end
    end
end
endcase.

```

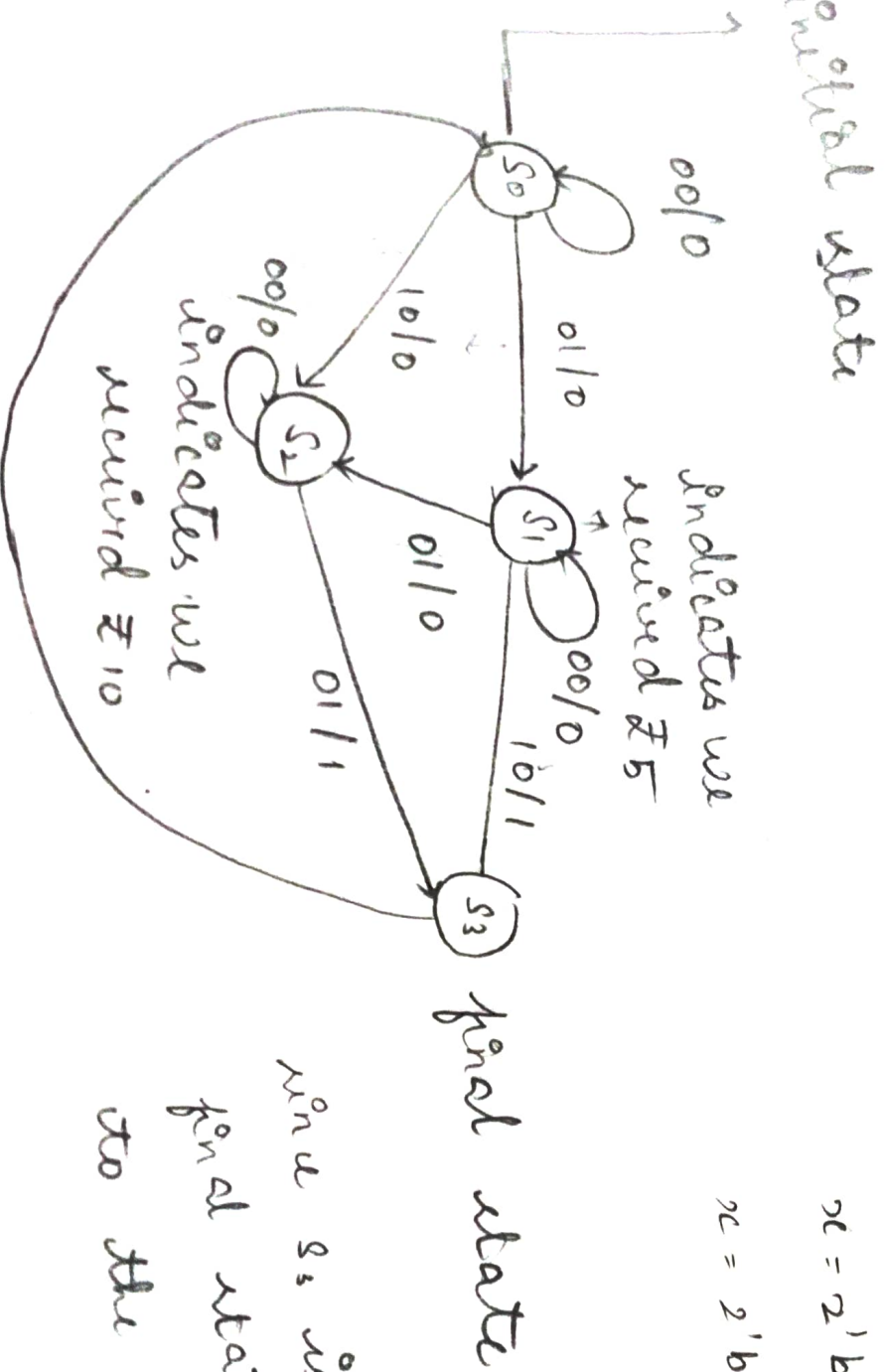
```

always @ (posedge clk)
begin
    if (rst)
        ps = 0;
    else
        ps = ns;
    end
end module

```

Vending machine

state diagram.



since S_3 is the final state it goes to the state S_0

$x = 2'b00 \rightarrow$ no coin is inserted
 $x = 2'b01 \rightarrow$ 5 rupee coin
 $x = 2'b10 \rightarrow$ 10 rupee

state table
 Q_1, Q_0

Q_1^+, Q_0^+

Present state	Input	Next state	y
S_0 00	00	S_0 00	0
S_0 00	01	S_1 01	0
S_0 00	10	S_2 10	0
S_1 01	00	S_1 01	0
S_1 01	01	S_2 10	0
S_1 01	10	S_0 00	0
S_2 10	00	S_2 10	0
S_2 10	01	S_0 00	1

K-map for Q_1^+

Q_1, Q_0	$x_1 x_0$ 00	01	11	10
00				①
01		①		
11				
10	①			

$$Q_1^+ = \overline{Q_1} \overline{Q_0} x_1 \overline{x_0} + \overline{Q_1} Q_0 \overline{x_1} x_0 + Q_1 \overline{Q_0} \overline{x_1} \overline{x_0}$$

K map for Q_0^+

$Q_1 Q_0$	$x_1 x_0$			
	00	01	11	10
00		1		
01	1			
11				
10				

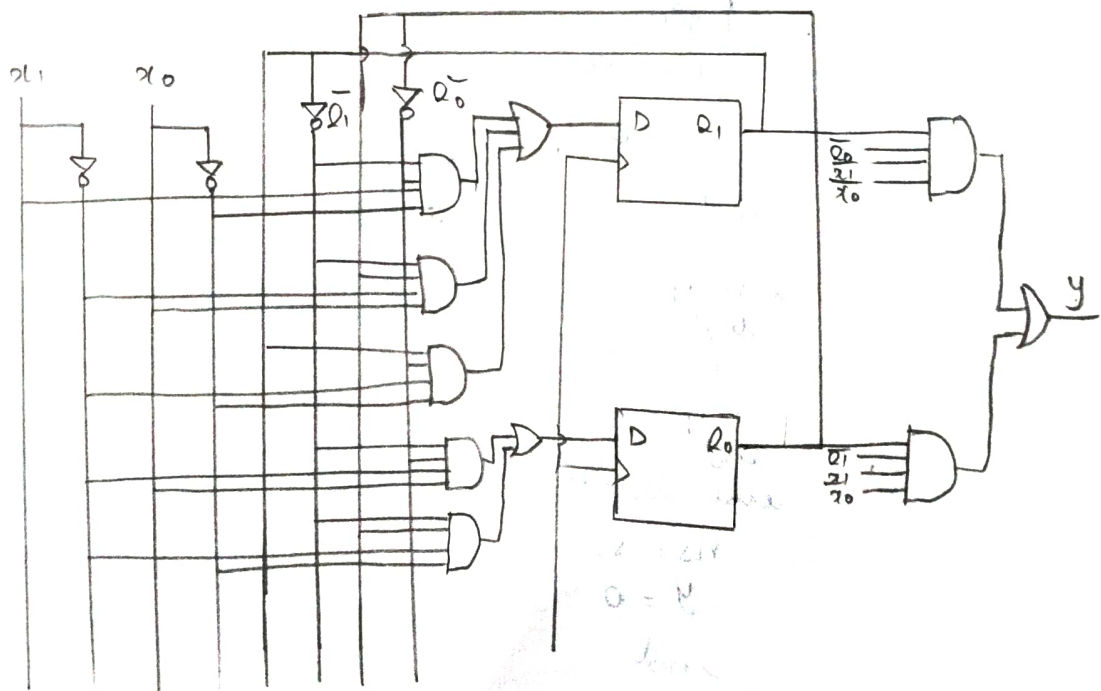
$$Q_0^+ = \bar{Q}_1 \bar{Q}_0 \bar{x}_1 x_0 + \bar{Q}_1 Q_0 \bar{x}_1 \bar{x}_0$$

K-map for y .

$Q_1 Q_0$	$x_1 x_0$			
	00	01	11	10
00				
01				1
11				
10		1		

$$y = Q_1 \bar{Q}_0 \bar{x}_1 \bar{x}_0 + \bar{Q}_1 Q_0 x_1 \bar{x}_0$$

Logic diagram.



Verilog code

```
module vml(input clk, sel, input [1:0] x,  
           output y);
```

```
reg [1:0] ps, ns;
```

```
parameters s0 = 2'b00, s1 = 2'b01, s2 = 2'b10
```

```
always @(posedge clk)
```

```
begin
```

```
if (sel)
```

```
    ps <= 2'b00;
```

```
else
```

```
    ps <= ns;
```

```
end
```

```
always @(ps, x)
```

```
begin
```

```
case (ps)
```

```
    s0: begin
```

```
        if (x == 2'b00)
```

```
            begin
```

```
                ns = s0
```

```
                y = 0
```

```
            end
```

```
        else if
```

```
            begin
```

```
                ns = s1
```

```
                y = 0
```

```
            end
```

```
        else begin
```

```
            ns = s2
```

```
            y = 0
```

```
        end
```

```
    end
```

s₁: begin

if (x == z' b 01)

begin
ns = s₁

end
y = 0

else if

begin
ns = s₀

end
y = 1

else begin

ns = s₂

y = 0

end
end

s₂: begin

if (x == z' b 10)

begin

ns = s₂

y = 0

end

else begin

ns = s₀

y = 0

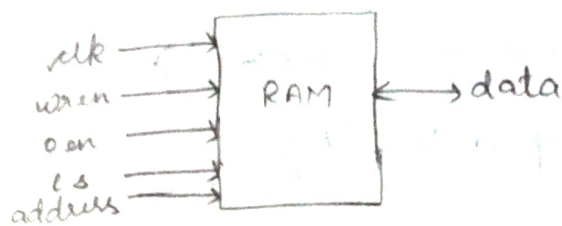
end

end

endcase

end

Single Port - RAM



single port - RAM means the both read write operation performed through the same port.

data should act as the bidirectional port acts as both input and output.

Verilog module

```
module ram(clk, rst, wren, oen, cs, data);
```

```
input clk, rst; input [5:0] address;
```

```
input wren, oen, cs;
```

```
inout [7:0] data; reg [7:0] temp_data;
```

```
reg [7:0] mem [0:7];
```

↓ ↓
 width of depth of the
 the memory memory

maximum 8 bit
 data
 can be written

```
always@ (posedge clk)
```

```
begin
```

```
if (wren && cs)
```

```
mem[address] = data;
```

```
end
```

```
Read operation.
```

```
always@ (posedge clk)
```

```
begin
```

```
if (oen && cs)
```

```
begin
```

```

temp_data := mem[address];
end
assign data = (cs && !w.en && seen)?
temp_data : hz;
endmodule

```

Single port ROM.

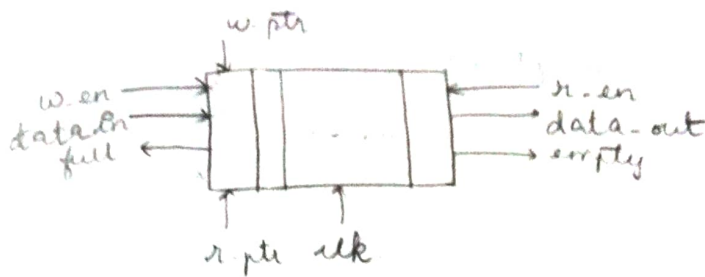
```

module rom (clk, o-en, cs, data)
input clk;
input o-en, cs;
output data; input [3:0] address;
reg [1:0] mem[3:0];
always @ (posedge clk)
begin
if (o-en && !cs)
begin
data <= mem[address];
end
end
endmodule.

```

Synchronous FIFO

* Both read and write operation is performed using same clk cycle is called synchronous FIFO simultaneously.



FIFO should not be full to write a data. it should be vacant.

To read a data the FIFO should not be empty.

Initially w_ptr and r_ptr are at same place that is empty, or at 0

$$\text{empty} = w_ptr == r_ptr$$

$$\text{full} = w_ptr + 1 == r_ptr$$

```
module fifo (clk, rst, w-en, r-en, data-in, data-out,
             full, empty);
```

```
    input clk, rst;
```

```
    input w-en, r-en;
```

```
    input [7:0] data-in;
```

```
    output full, empty;
```

```
    reg [2:0] w_ptr, r_ptr;
```

```
    reg [7:0] mem [0:7];
```

```
always @ (posedge clk)
```

```
begin
```

```
    if (rst)
```

```
        w_ptr <= 00;
```

```
    else begin if (!full) begin
```

```
        mem[w_ptr] <= data-in;
```

w_ptr <= w_ptr + 1

end

end

end

always @(posedge clk)

begin

if (set)

r_ptr <= 0;

else

begin

if (!empty)

begin

data_out <= mem[r_ptr]

r_ptr <= r_ptr + 1;

end

end

end

assign full = w_ptr + 1 == r_ptr;

assign empty = w_ptr == r_ptr;

endmodule.

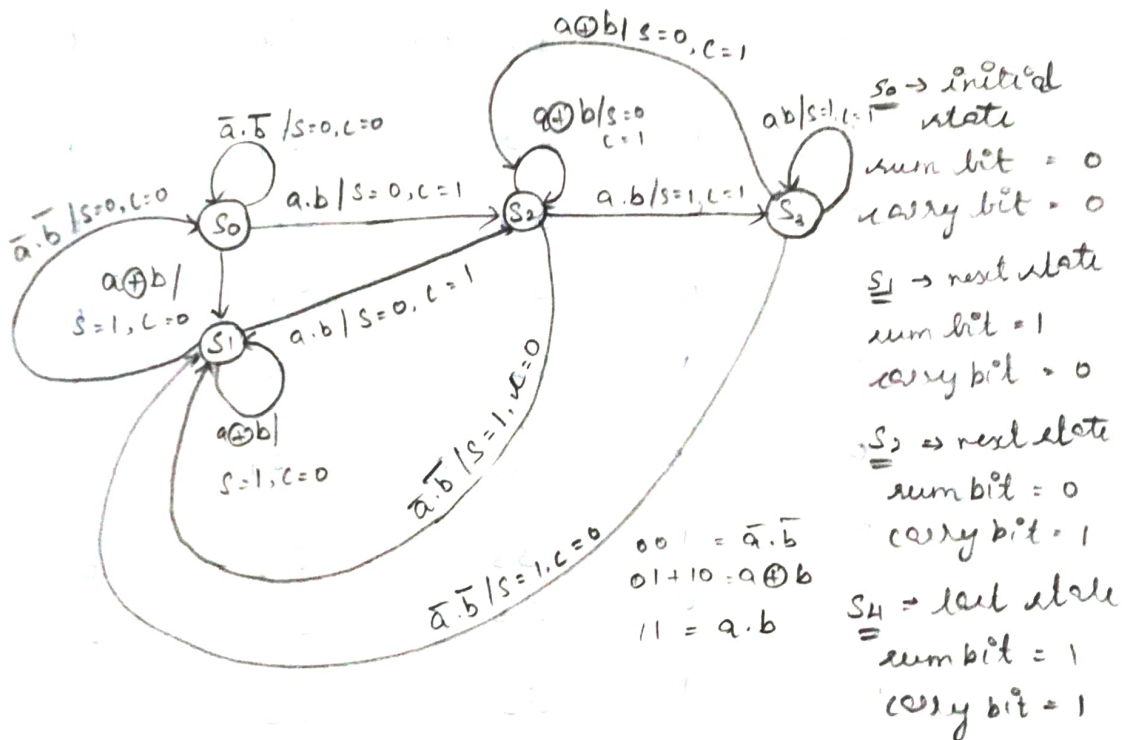
FSM Modeling of Serial Adder.

Serial Adder.

the input for the adder is serial.
bit by bit for every clk pulse one
bit is given, adder calculate the sum
and carry.

$$\begin{array}{r} a \quad 0 \quad 1 \quad 0 \quad 1 \\ b \quad 1 \quad 1 \quad 0 \quad 1 \\ \hline \end{array}$$

input a, b, clk
output sum, carry



state

state table.

Input		Present state	Next state	Output	
a	b			s	c
0	0	S ₀ 00	S ₀	0	0
0	1	S ₀ 00	S ₁	1	0
1	0	S ₀ 00	S ₁	1	0
0	0	S ₁ 01	S ₀	0	0
0	1	S ₁ 01	S ₁	1	0
1	0	S ₁ 01	S ₁	1	0
1	1	S ₁ 01	S ₂	0	1
0	0	S ₂	S ₁	1	0
0	1	S ₂	S ₂	0	1
1	0	S ₂	S ₂	0	1
1	1	S ₂	S ₃	1	1
0	0	S ₃	S ₁	1	0
0	1	S ₃	S ₂	0	1
1	0	S ₃	S ₂	0	1
1	1	S ₃	S ₃	1	1

Verilog code

```
module sa (input [1:0] a, b, input clk, rst
          output s, c);
```

```
reg [1:0] ps, ns;
```

```
always @(posedge clk)
begin
```

```
    if (out)
        ps = 2'b00;
    else
        ps = ns;
    end
```

```
always @(ps, a, b)
begin
```

```
    case (ps)
        00: if (~a & ~b)
            begin
                ns = 2'b00;
                s = 1'b0;
                c = 1'b0;
            end
        else if (~a & b)
            begin
                ns = 2'b01;
                s = 1'b1;
                c = 1'b0;
            end
        else if (a & ~b)
            begin
                ns = 2'b10;
                s = 1'b0;
                c = 1'b1;
            end
        else if (a & b)
            begin
                ns = 2'b11;
                s = 1'b1;
                c = 1'b1;
            end
    end
```

20/08
class

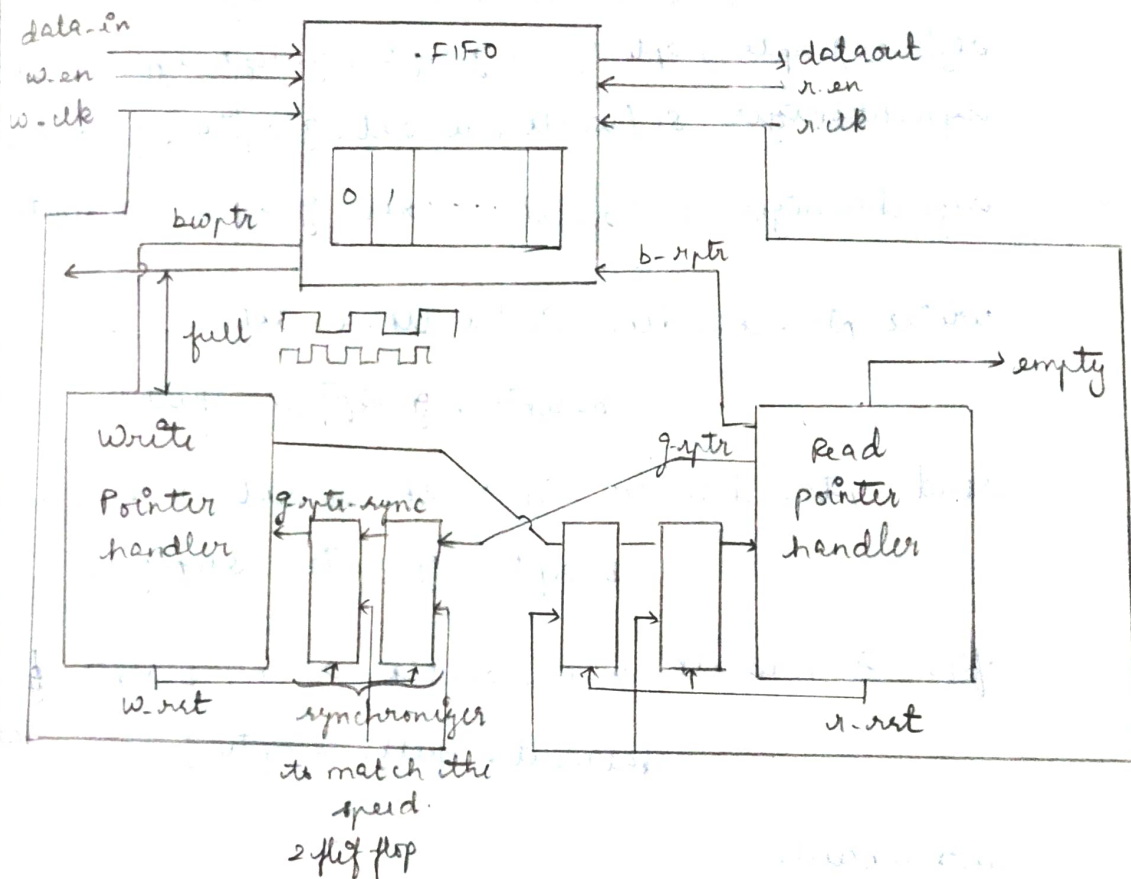
Asynchronous FIFO



[to avoid metastable state
grey code is used]

[metastable state
is caused by
normal code]

Block diagram.



Normal bit grey code

0000	0000
0001	0001
0010	0011
0011	0010
0100	0110
0101	0111
0110	0101
0111	0100

(normal bit
changes for each
clk pulse, so it
leads to meta
stable state)

Verilog code

Top module

```
module asyn_fifo (input w_clk, r_clk, w_en, r_en,  
                  w_out, r_out, input [3:0] data_in,  
                  output full, empty,  
                  output [3:0] data_out);  
    reg [3:0] g_wptr, g_rptr, g_wptr_sync, g_rptr_sync, b_wptr,  
    synchronizer s1 (w_clk, w_out, g_wptr, g_wptr_sync),  
    synchronizer s2 (r_clk, r_out, g_rptr, g_rptr_sync),  
    write_ptr_handler s3 (w_clk, w_out, w_en, g_wptr,  
                           b_wptr, g_wptr, full),  
    read_ptr_handler s4 (r_clk, r_out, r_en, g_rptr,  
                           b_rptr, g_rptr, empty);  
    fifo s5 (w_clk, w_en, r_clk, r_en, b_wptr, b_rptr,  
             data_out, full, empty); data_in
```

and module

synchronizer s.

module synchronizer

```
    input clk, rst,  
    output [3:0] d_in,  
    output [3:0] d_out;  
    always @ (posedge clk) 20  
begin  
    if (rst)  
begin
```

$q'_0 = 0$

$d_{out} = 0$

end else

$q_0 = d_{in};$
 $d_{out} = q_0;$

end

end module

Top module

```
module top (input w-clk, r-clk,
```

```
        w-en, r-en, w-out, r-out,
```

```
        module fifo 8b1
```

```
        input w-clk, w-en, r-clk, r-en, input [3:0] data-in,
```

```
        b-wptr, b-rptr, full, empty
```

```
        output reg [3:0] data);
```

```
        always @ (posedge w-clk)
```

```
        begin
```

```
            if (w-en & !full)
```

```
                fifo[b-wptr] <= data-in;
```

```
            and.
```

```
        always @ (posedge r-clk)
```

```
        begin
```

```
            if (r-en & !empty)
```

```
                data <= fifo[b-rptr];
```

```
            end
```

```
        endmodule.
```

```
module read_ptr_handler (input r-clk, r-out,
```

```
        r-en, input [3:0] g-wptr, g-rptr,
```

```
        output reg [3:0] b-wptr, g-wptr,
```

```
        output reg empty);
```

always @ (posedge x_clk)

begin

if (!x_ack)

begin

b_rptn = 0

g_rptn = 0

end

else

begin

b_rptn = b_rptn - next;

g_rptn = g_rptn - next;

end

end

always @ (posedge v_clk)

begin

if (!x_ack)

empty = 1;

else

empty = x_empty

011

end

always b_rptn - next = b_rptn + (x_en & !empty);

always g_rptn - next = (b_rptn - next >> 1) * (b_rptn - next

x_empty = (g_rptn - sync == g_rptn - next);
endmodule.

module with_fa_handle_input_walk,

csut, w_en, input [3:0] g_rptn_sync,

output reg [3:0] b_rptn, g_rptn,
output reg full;

reg [3:0] b_rptn - next, g_rptn - next, v_empty

$$\begin{array}{r} 0110 \\ 1101 \\ \hline 1011 \end{array}$$

always @ (posedge w-clk)

begin

if (!w-ack)

begin

b-wptr <= 0;

g-wptr <= 0;

end

else begin

b-wptr <= b-wptr - next;

end g-wptr <= g-wptr - next;

always @ (posedge w-clk)

begin

if (!w-ack)

full <= 0;

else

full <= w-full;

end

assign b-wptr - next = b-wptr + (w-ack & !full);

assign g-wptr - next = (b-wptr - next > 1) ? (b-wptr - next);

assign w-full = (g-wptr - next == g-wptr - wptr);

endmodule.